

Tessera

Gestão de clubes e bilheteira digital

APRESENTAÇÃO DE PROGRESSO

Grupo 31

Bruno Pinto, n.º 48064

Daniel Cabaça, n.º 48070

Orientador: José Faustino

O que é o Tesserá?

Motivação, contexto e visão geral da solução

O PROBLEMA

Clubes de futebol de divisões inferiores em Portugal enfrentam desafios operacionais por falta de ferramentas digitais.

- * Bilhetes em papel — custos de impressão, risco de falsificação, dificuldade no controlo de entradas
- * Ficha técnica preenchida à mão — erros, atrasos e dificuldade de consulta
- * Sem rastreio de vendas, sem historico digital, sem análise de dados

A SOLUÇÃO

Plataforma digital para gestão completa de bilheteira e ficha técnica dos jogos.

- * Aplicação web (React SPA) para administradores e adeptos
- * Aplicação Android para validação de bilhetes e ficha técnica
- * Bilhetes digitais com QR code (MB WAY, multibanco, dinheiro)
- * Registo de ocorrências do jogo em tempo real
- * Histórico digital de jogos e relatórios de bilheteira

Caso de uso real: *SU 1.º Dezembro — clube parceiro disponível para cooperar na elaboração da solução*

Requisitos & Perfis de utilizador

Três perfis distintos com necessidades diferentes

ADM

Administrador

- Gestão de clubes e equipas
- Gestão de jogadores e plantéis
- Criação de jogos e eventos
- Configuração de bilheteira
- Consulta de relatórios e estatísticas

STF

Staff

- Validação de bilhetes (QR code)
- Preenchimento da ficha técnica
- Registo de ocorrências em tempo real (golos, cartões, substituições)
- Operações no dia de jogo

FAN

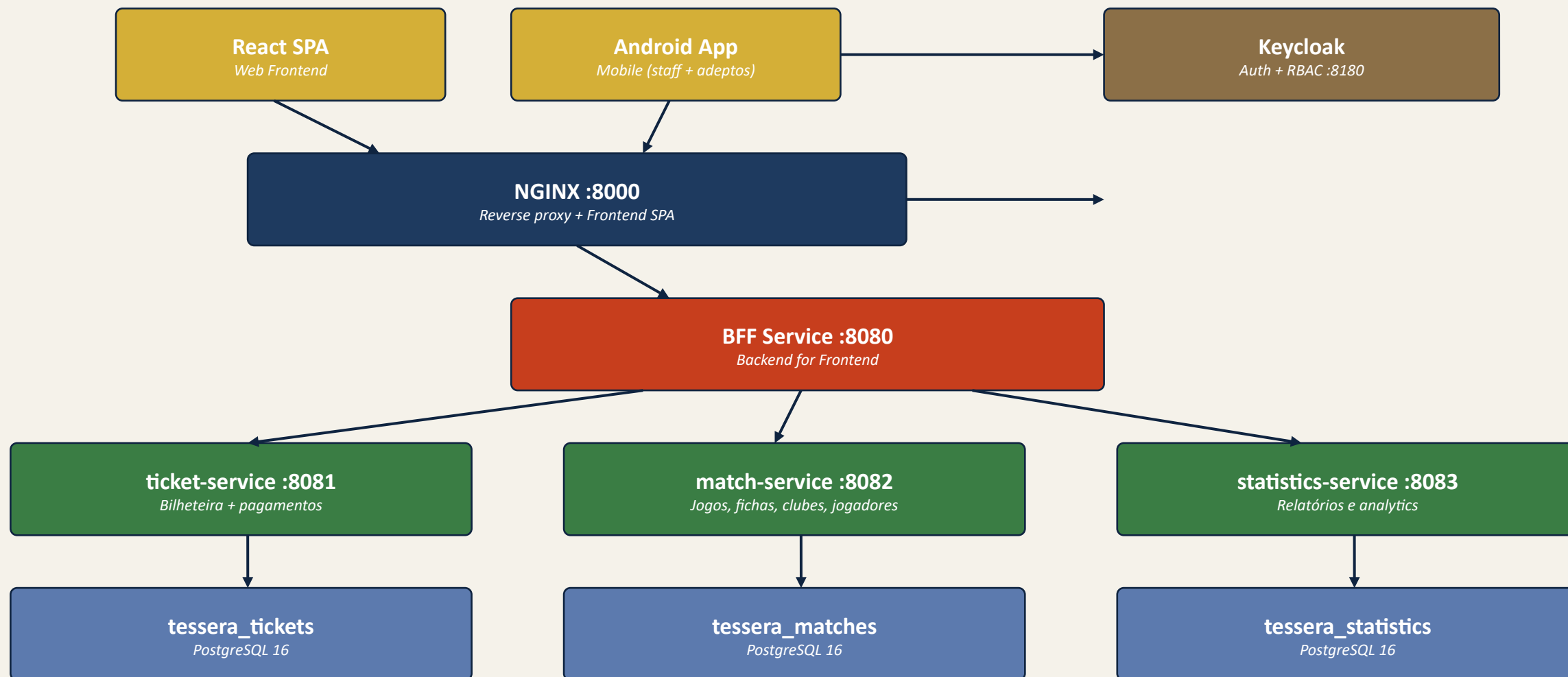
Adepto

- Compra de bilhetes digitais
- Pagamento via MB WAY
- Visualização de bilhete (QR)
- Consulta de jogos e resultados
- Acesso a fichas técnicas

Requisitos não funcionais: *Desempenho (validação rápida) - Segurança (RBAC + HTTPS) - Disponibilidade - Usabilidade*

Arquitetura do sistema

Microserviços, BFF e isolamento de dados



Cada serviço com BD própria, deploy independente, comunicação via REST síncrono entre BFF e serviços internos.

Microsserviços

Responsabilidades e domínio de cada serviço

BFF Service

:8080 (sem BD)

Backend for Frontend

Ponto de entrada único para a API. Adapta respostas a cada cliente, agrega chamadas a múltiplos serviços.

Ticket Service

:8081 tessera_tickets

Bilheteira

Eventos, compra de bilhetes (PENDING → PAID → VALIDATED), validação por QR code, pagamentos.

Match Service

:8082 tessera_matches

Atividade desportiva

Clubes, equipas, jogadores, jogos, fichas técnicas e ocorrências (golos, cartões, substituições).

Statistics Service

:8083 tessera_statistics

Análise e relatórios

Serviço de leitura. Relatórios de bilheteira, estatísticas de jogos e jogadores, histórico.

Decisão: Clubes, equipas e jogadores vivem dentro do match-service (em vez de serviço separado) — são consumidos quase exclusivamente pela ficha técnica.

Stack tecnológico

Tecnologias selecionadas para cada componente do sistema

BACKEND	Spring Boot + Kotlin	3.4.4 / 1.9.25
MOBILE	Android (Kotlin + Compose)	—
MIGRAÇÕES BD	Flyway	—
REVERSE PROXY	NGINX (Alpine)	—
BUILD TOOL	Gradle (Kotlin DSL)	8.13
API SPEC	OpenAPI 3.1 + Redocly	—

FRONTEND WEB	React + TS + Vite + Tailwind	—
BASE DE DADOS	PostgreSQL (Alpine)	16
AUTENTICAÇÃO	Keycloak (OIDC + OAuth2)	26.0
CONTAINERS	Docker + Docker Compose	—
JDK	Eclipse Temurin	21
MENSAGENS	RabbitMQ	3.13

O que está feito - Infraestrutura

Setup completo de Docker, NGINX e Keycloak

OK

Docker Compose

9 containers orquestrados — 4 serviços backend, 3 BDs, NGINX, Keycloak. Rede interna com DNS, volumes persistentes, health checks.

OK

NGINX como reverse proxy

Routing `/api/*` → BFF, `/realms/*` + `/admin/*` → Keycloak, `/*` → React SPA. Frontend embutido na imagem (multi-stage build).

OK

Keycloak — autenticação e autorização

Realm 'tessera' com 3 roles (admin, staff, fan), 3 clients (web, android, bff) com PKCE, 3 utilizadores de teste, importação automática.

OK

Scripts de automação (PowerShell)

`buildAll.ps1`, `start.ps1`, `stop.ps1`, `reset.ps1` e scripts individuais por serviço (`reset_XXX_service.ps1`) para builds e gestão de containers.

OK

Documentação

6 documentos Markdown — architecture, docker, nginx, keycloak, scripts, getting-started — atualizados à medida que o projeto evolui.

O que está feito - Backend

Migração de monólito para microserviços e proxy BFF

PONTO DE PARTIDA

Backend monolítico

Spring Boot + Kotlin

Apenas 2 entidades:

- * Event
- * Ticket

Endpoints básicos:

- * POST /api/tickets
- * POST /api/tickets/validate

ARQUITETURA ATUAL

bff-service

Proxy + agregação

:8080

ticket-service

Bilhetes digitais, QR, pagamento, validação, eventos RabbitMQ

:8081

match-service

Clubes, equipas, jogadores, jogos + ficha técnica e ocorrências

:8082

statistics-service

Read-side de relatórios + vendas (consumidor RabbitMQ)

:8083

Cada serviço: BD própria + Flyway migrations + Dockerfile + build script + reset script

O que está feito - Frontend Web

Implementação a partir do handoff do Claude Design

OK

Design system & tokens

Tailwind v4 (CSS-first) + shadcn/ui (new-york style). Paleta OKLCH com 4 cores semânticas (pending, paid, validated, cancelled). Dark mode via classe .dark. Tokens em src/index.css.

OK

Catálogo público de jogos

/events com filtros (Todos / Em casa / Fora / Esta semana) + search. /events/:id com hero, tabela de preços (Normal/Sócio) e admin sales card. Crests geométricos por clube.

OK

Fluxo de compra em 3 passos

Modal stepper (shadcn Dialog): escolher tier → método de pagamento (MBWAY/Cartão/Dinheiro) → confirmação com QR. Integrado com POST /tickets e POST /tickets/{id}/pay.

OK

"Os meus bilhetes" + QR

/tickets/mine agrupado por estado (Prontos, Aguarda pagamento, Histórico). QR renderizado client-side com qrcode.react sobre o code UUID do bilhete. Card com stub perfurado tipo bilhete real.

OK

Validação no portão (staff)

/validate mobile-first com 3 estados visuais (idle/success/reject), ligado ao POST /tickets/validate. Verde com check para PAID→VALIDATED, vermelho com X para já-usado / inválido / 404.

O que está feito - Especificação da API

Contrato OpenAPI 3.1 como fonte de verdade

Abordagem contract-first

A especificação OpenAPI é o contrato vinculativo entre os 3 consumidores (web, android, BFF) e a backend. A implementação deriva da spec.

- 1 Definir endpoints e schemas em YAML (docs/api/)
- 2 Validar com Spectral (linter de OpenAPI)
- 3 Gerar documentação HTML interativa (Redocly)
- 4 Servir Swagger UI no BFF (/api/docs)
- 5 Gerar clientes tipados (TypeScript + Kotlin)

RECURSOS DA API

Match-service

- * /clubs - clubes
- * /teams - equipas
- * /players - jogadores
- * /venues - estádios
- * /matches - jogos
- * /matches/{id}/sheet - ficha técnica

Ticket-service

- * /tickets · /tickets/mine
- * /tickets/{id}/pay · /tickets/validate
- * /events catalog (admin)

Statistics-service

- * /stats/match-sheets
- * /stats/sales/summary · /by-match
- * /stats/sales/range?from=&to=

Convenções: /api/v1 - camelCase - ISO 8601 - Problem Details (RFC 7807) - JWT Bearer (Keycloak)

Modelo de dados

Entidades principais por contexto (BD por serviço)

MATCH-SERVICE

Club

id, name, foundedYear, crestUrl

Team

id, clubId, category

Player

id, teamId, name, position, shirtNumber

Venue

id, name, capacity, address

Match

id, homeTeamId, awayTeamId, kickoffAt, status

MatchSheet

id, matchId, locked

Occurrence

id, minute, type (GOAL/CARD/SUB)

TICKET-SERVICE

Event

id, matchId, name, priceNormal, priceSupporter, status

Ticket

id, eventId, code (UUID), price, status, ownerSub, validatorSub, paymentMethod

Lifecycle:

PENDING

(criado, aguarda pagamento)

PAID

(pagamento confirmado)

VALIDATED

(usado à entrada)

STATISTICS-SERVICE

MatchSummary

matchId, season, status, kickoff, scores, refereeName

LineupSnapshot

matchId, playerId, teamId, role

OccurrenceSnapshot

id, matchId, minute, type (GOAL/CARD/SUB)

TicketSale

ticketId, eventId, matchId, price, paidAt, validatedAt

Tabelas read-only

alimentadas via

RabbitMQ events

(implementado)

Demonstração

Sistema funcional end-to-end

Fluxo completo a correr localmente

```
> .\scripts\run\start.ps1

[OK] Building all services... (buildAll.ps1)
bff-service      : SUCCESS
ticket-service   : SUCCESS
match-service    : SUCCESS
statistics-service : SUCCESS

[OK] Starting Docker containers...
9 containers running on tessera-net

All services are up!
App:      http://localhost:8000
BFF API:  http://localhost:8000/api
Keycloak: http://localhost:8180
```

Demonstração em vivo:

- * Frontend React acessível em <http://localhost:8000>
- * Login via Keycloak com 3 utilizadores diferentes (admin, staff, adepto) — JWT contém roles
- * Compra de bilhete: SPA → NGINX → BFF → ticket-service → PostgreSQL
- * Validação de bilhete por UUID — alteração de estado PAID → VALIDATED
- * Consola de admin do Keycloak para gestão de utilizadores e roles

Desafios técnicos enfrentados

Problemas reais e como foram resolvidos

Problema: Build do gradlew falhava em Alpine Linux

Causa: O BusyBox sh não interpreta corretamente os JVM args do gradlew (DEFAULT_JVM_OPTS com quotes aninhadas)

Solução: Adotamos abordagem runtime-only: build do JAR na máquina local, Dockerfile apenas COPY do JAR pré-compilado

Problema: NGINX retornava 404 em /api/ e /realms/

Causa: A imagem base do NGINX inclui um default.conf com server block na porta 80 que interceptava os pedidos antes da nossa configuração

Solução: Adicionamos RUN rm /etc/nginx/conf.d/default.conf no Dockerfile

Problema: JWT do Keycloak sem claim "sub"

Causa: O realm-export não tinha mapper para "sub"; jwt.subject vinha null e rebentava NPE no ticket-service

Solução: Adicionamos protocol mapper "sub" no realm + fallback chain (sub → preferred_username) no helper userIdOf()

Problema: Keycloak 26 sem prefixo /auth/

Causa: Ao contrário das versões < 25, o Keycloak 26 serve diretamente em /realms/, /admin/, /resources/

Solução: Atualizamos a configuração NGINX com locations explícitos para cada caminho do Keycloak

Problema: CORS no browser ao chamar a API

Causa: Frontend chamava http://localhost:8080 diretamente, gerando cross-origin request

Solução: Mudamos o frontend para path relativo /api — todos os pedidos passam pelo NGINX

O que falta fazer

Funcionalidades em falta e próximos passos

BACKEND

- * Integração real com gateway SIBS (webhook callback)
- * Dead-letter queue + retry nos consumers RabbitMQ
- * Tracing distribuído (traceld nos eventos)
- * Reports avançados (top scorers, attendance)
- * Refund flow (VALIDATED → REFUNDED por admin)
- * Limite de capacidade por evento (row-lock)
- * Testes (unitários + integração)

FRONTEND + MOBILE

- * Painéis admin: clubes, equipas, jogadores (CRUD)
- * Substituir mocks por queries reais ao API
- * Camera scanning real no /validate (jsQR / zxing)
- * App Android: validação de QR (staff)
- * App Android: ficha técnica em tempo real
- * App Android: visualização de bilhetes (adeptos)
- * Geração de cliente TypeScript a partir do OpenAPI

OPS + DELIVERY

- * Configurar HTTPS (certificado SSL)
- * Pipeline CI/CD
- * Substituir BD H2 do Keycloak por PostgreSQL
- * Logs centralizados
- * Deploy em ambiente de produção
- * Documentação de utilizador final
- * Validação com SU 1.º Dezembro

Próximos passos & Planeamento

FASE 1

Backend match-service ✓ - *Clubes, equipas, jogadores, jogos, ficha técnica, ocorrências, auto-lock.*

FASE 2

Bilhetes digitais (ticket-service) ✓ - *Ownership por JWT, QR, pay flow, validação. Gateway SIBS real fica para depois.*

FASE 3

Statistics + RabbitMQ events ✓ - *Read-side de relatórios e vendas via eventos AMQP (CQRS event-driven).*

FASE 4

Frontend + Android (em curso) - *SPA P3 implementada do handoff Claude Design. Wire-up API real + Android em curso.*

FASE 5

SIBS real + Deploy + SU 1.º Dezembro - *Integração real com gateway de pagamento, HTTPS, CI/CD, deploy + validação com o clube parceiro.*

Perguntas?

Obrigado pela atenção - Bruno Pinto & Daniel Cabaça - Grupo 31

github.com/bruno-pinto-git/tessera